

Министерство образования Республики Беларусь  
Учреждение образования  
«Белорусский государственный университет информатики и  
радиоэлектроники»

Кафедра электронных вычислительных машин

Лабораторная работа №1  
«Последовательный порт»

Выполнил:  
Студент группы 150503  
Шарай П. Ю.

Проверил:  
к. т. н., доцент  
Одинец Д.Н.

Минск, 2023

## 1. Постановка задачи

Разработать программный модуль реализации процедуры передачи (приёма) байта информации через последовательный интерфейс.

Программа должна демонстрировать программное взаимодействие с последовательным интерфейсом с использованием следующих механизмов:

1. Прямое взаимодействие с портами ввода-вывода (write, read)
2. Использование BIOS прерывания 14h
3. Работа с COM-портом через регистры как с устройствами ввода-вывода.

## 2. Алгоритм

Для решения поставленной задачи программу необходимо разделить на несколько подпрограмм, реализующих некоторые функции:

1. Инициализация порта
2. Запись байта в порт
3. Чтение информации из порта
4. Вывод результата на экран

## 3. Листинг программы

Далее приведены листинги программ, представляющих различные способы реализации приема (передачи) байта информации через последовательный интерфейс.

### 3.1. Прямое взаимодействие с портами ввода-вывода (write, read)

```
#include <windows.h>
#include <iostream>
using namespace std;

// объявление портов
HANDLE COMP1;
LPCTSTR Port_Name1 = L"COM1";
HANDLE COMP2;
LPCTSTR Port_Name2 = L"COM2";

void read_from_com()
{
    DWORD size;
    char rec_symbol;
    ReadFile(COMP2, &rec_symbol, 1, &size, 0);
    if (size > 0)
    {
        cout << rec_symbol;
    }
}
```

```

int main()
{
    //инициализация портов
    COMP1 = ::CreateFile(Port_Name1, GENERIC_WRITE, 0, 0, OPEN_EXISTING,
FILE_ATTRIBUTE_NORMAL, 0);
    COMP2 = ::CreateFile(Port_Name2, GENERIC_READ, 0, 0, OPEN_EXISTING,
FILE_ATTRIBUTE_NORMAL, 0);

    if (COMP1 == INVALID_HANDLE_VALUE) // ошибка функции CreateFile
    {
        if (GetLastError() == ERROR_FILE_NOT_FOUND) // если порт не
существует
        {
            cout << "COM port doesn't exist!" << endl;
        }
        cout << "Error" << endl;
    }

    DCB serial_params = { 0 };
    serial_params.DCBlength = sizeof(serial_params); // длина структуры в
байтах
    if (!GetCommState(COMP1, &serial_params))
    {
        cout << "State error" << endl;
    }
    serial_params.BaudRate = CBR_9600; // скорость устройства связи (9600
бит в секунду)
    serial_params.ByteSize = 8; // количество битов в байтах, переданных
и полученных
    serial_params.StopBits = ONESTOPBIT; // количество используемых
стоповых бит
    serial_params.Parity = NOPARITY; // схема паритета, которая будет
использоваться
    if (!SetCommState(COMP2, &serial_params))
    {
        cout << "Error setting serial port state" << endl;
    }

    char data = 'A';
    DWORD data_size = sizeof(data);
    DWORD written_bytes;

    BOOL Ret = WriteFile(COMP1, &data, data_size, &written_bytes,
NULL); // запись информации в порт

    cout << "Bytes in string: " << data_size << ';' << " bytes sended: "
<< written_bytes << endl;

    read_from_com(); // чтение из порта

    return 0;
}

```

## 3.2 Использование BIOS прерывания 14h

```

.model small
.stack 100h

.data
Error_Write db "Write error!",0Dh,0Ah,'$'
Error_Read db "Read error!",0Dh,0Ah,'$'
Information db "Byte sent: $"

.code

```

```

jmp start

Init_COM1 proc ; инициализация порта
    xor ax,ax
    mov al,10100011b
    mov dx,0
    int 14h
    ret
Init_COM1 endp

IsWrite_COM1 proc ; запись информации
    mov al,'A'
    mov ah,1
    mov dx,0
    int 14h
    test al,80h
    jnz NoWrite
    ret
IsWrite_COM1 endp

NoWrite proc ; запись не была проведена успешно
    mov ah,9
    mov dx,offset Error_Write
    add dx,2
    int 21h
    ret
NoWrite endp

IsRead_COM2 proc ; чтение информации
    mov ah,2
    mov dx,1
    int 14h
    test al,80h
    jnz NoRead
    ret
IsRead_COM2 endp

NoRead proc ; информация не была прочитана
    mov ah,9
    mov dx,offset Error_Read
    add dx,2
    int 21h
    ret
NoRead endp

Output proc ; вывод результата на экран
    mov ah,02h
    mov dl,al
    int 21h
    ret
Output endp

Exit proc ; завершение работы
    mov ax,4C00h
    int 21h
    ret
Exit endp

start:
    call Init_COM1
    call IsWrite_COM1
    mov al,'e'
    call IsRead_COM2
    push ax

```

```

    mov ah,9
    mov dx,offset Information
    add dx,2
    int 21h

    pop ax
    call Output
    call Exit

end start

```

### 3.2. Работа с COM-портом через регистры как с устройствами ввода-вывода

```

.model small
.stack 100h

.data
Error_Write db "Write error",0Dh,0Ah,'$'
Error_Read db "Read error",0Dh,0Ah,'$'
Information db "Byte sent: $"
Data_Byte db 'A'
Data_Byte2 db ?

.code

Init_COM1 proc ; инициализация порта
    mov al,80h
    mov dx,3FBh ; line control register
    out dx,al

    mov dx,3F8h
    mov al,00h
    out dx,al
    mov al,0Ch
    mov dx,3F9h ; senior byte frequency divider
    out dx,al

    mov dx,3FCh
    mov al,00001011b
    out dx,al

    mov dx,3F9h
    mov al,0
    out dx,al
    ret
Init_COM1 endp

IsWrite_COM1 proc ; запись информации
    xor al,al
    mov dx,3FDh
    in al,dx
    test al,10h
    jnz NoWrite
    ret
IsWrite_COM1 endp

NoWrite proc ; запись не была проведена успешно
    mov ah,9
    mov dx,offset Error_Write
    int 21h
    ret
NoWrite endp

```

```

IsRead_COM2 proc ; чтение информации
    xor al,al
    mov dx,2FDh
    in al,dx
    test al,10b
    jnz NoRead
    ret
IsRead_COM2 endp

NoRead proc ; информация не была прочитана
    mov ah,9
    mov dx,offset Error_Read
    int 21h
    ret
NoRead endp

Send_Byte proc ; отправка информации
    mov dx,3F8h
    mov al,Data_Byte
    out dx,al
    ret
Send_Byte endp

Read_Byte proc ; чтение информации
    mov dx,2F8h
    in al,dx
    mov Data_Byte2,al
    ret
Read_Byte endp

Exit proc ; завершение работы
    mov ax,4C00h
    int 21h
    ret
Exit endp

start:
    mov ax,@data
    mov ds,ax
    call Init_COM1
    call IsWrite_COM1
    call Send_Byte
    mov al,2
    call IsRead_COM2
    call Read_Byte
    mov dx,offset Information
    mov ah,9
    int 21h
    mov ah,02h
    mov dl,Data_Byte2
    int 21h
    call Exit

end start

```

#### 4. Тестовые примеры

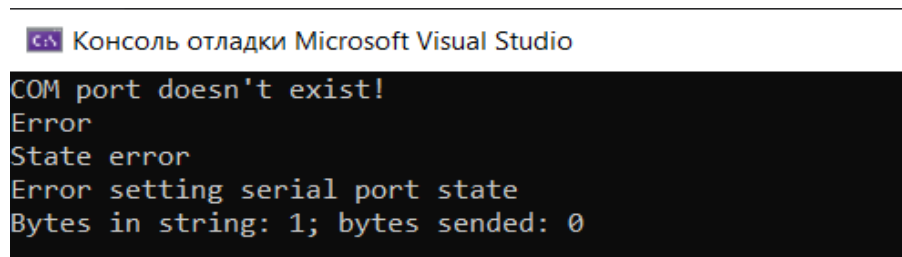


Рисунок 4.1 – Результат работы программы, взаимодействующей с портами ввода-вывода, при выключенной эмуляции COM-портов

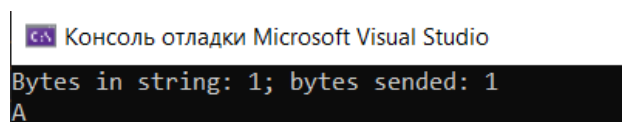


Рисунок 4.2 – Результат работы программы, взаимодействующей с портами ввода-вывода, при включенной эмуляции COM-портов

Byte sent: A

Рисунок 4.3 – Результат работы программы, использующей прерывание BIOS 14h

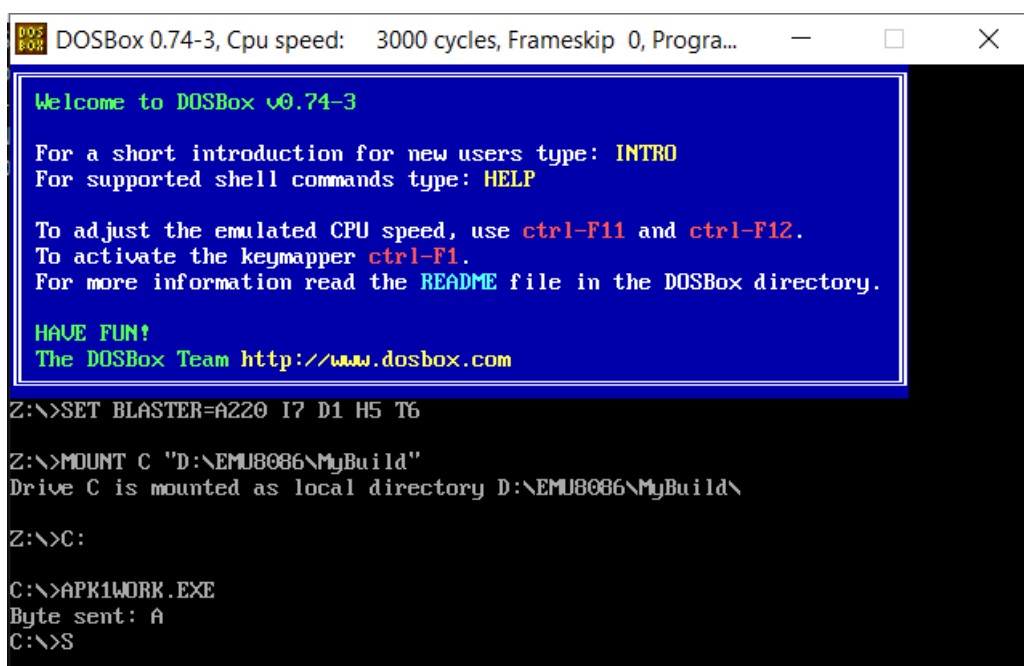


Рисунок 4.4 – Результат работы программы, взаимодействующей с COM-портом через регистры как с устройствами ввода-вывода

## 5. Заключение

В ходе лабораторной работы были разработаны программы, передающие 1 байт информации через последовательный порт с использованием различных механизмов.

Для эмуляции COM портов использовался Virtual Serial Port Driver, для эмуляции DOS используется Oracle VM Virtual Box на хосте 64-ех разрядной Windows 10. Для написания кода программ были использованы среды разработки Visual Studio 2019 и EMU8086.